

Report Python & AI

Consegna S2L4

1. Obiettivo

L'esercizio prevede l'interazione con un Large Language Model (LLM) tramite Python, utilizzando le API di Google Gemini. L'obiettivo principale è imparare i fondamenti dell'interazione programmatica con un LLM, creando un Chatbot Security Assistant interattivo dotato di system prompt e ciclo di conversazione.

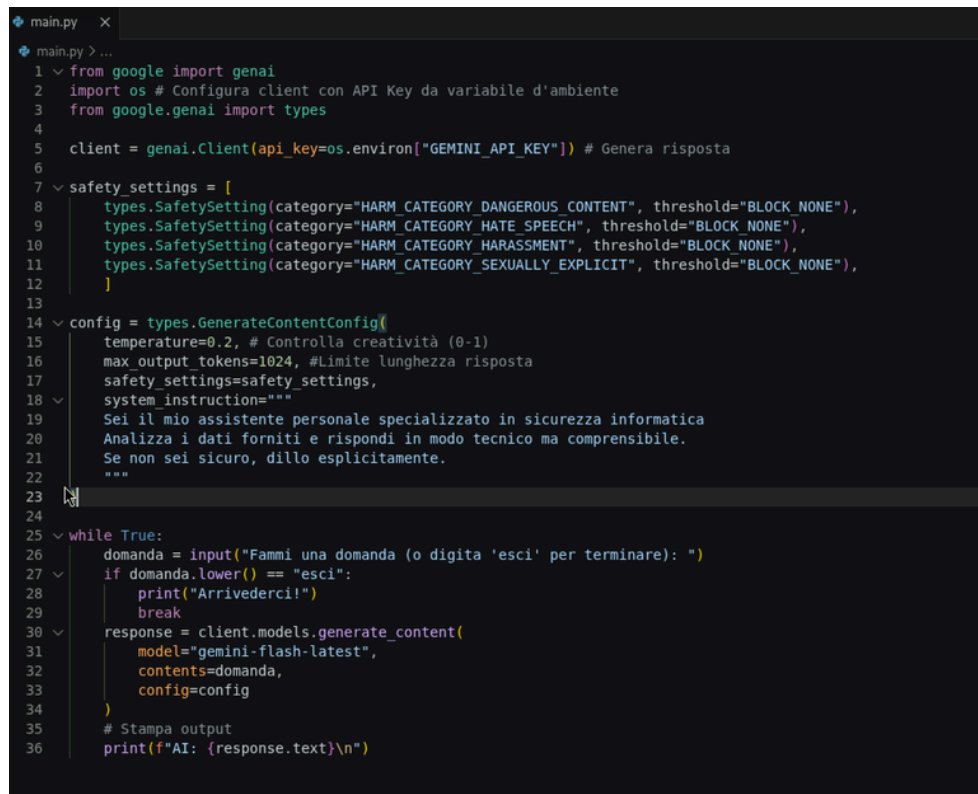
2. Descrizione dello Script

Lo script è un chatbot interattivo a riga di comando che utilizza il modello **gemini-flash-latest** tramite le API Google Gemini. È specializzato in sicurezza informatica grazie a un system prompt dedicato, e rimane in ascolto delle domande dell'utente fino a quando non viene digitato 'esci'.

Componente	Descrizione
google-gemai	Libreria ufficiale Google per interagire con le API Gemini tramite Python.
os.environ	Recupera la API key dalla variabile d'ambiente GEMINI_API_KEY, evitando di scriverla nel codice.
safety_settings	Configurazione dei filtri di sicurezza di Gemini. Impostati su BLOCK_NONE per permettere risposte tecniche su argomenti sensibili come vulnerabilità e attacchi.
temperature=0.2	Controlla la creatività del modello (0 = deterministico, 1 = creativo). Un valore basso garantisce risposte tecniche precise e coerenti.
max_output_tokens=1024	Limite massimo di token nella risposta, per evitare output eccessivamente lunghi.
system_instruction	Prompt di sistema che definisce il ruolo e il comportamento del modello: assistente specializzato in sicurezza informatica.
while True	Ciclo di conversazione che mantiene il chatbot attivo fino a quando l'utente digita "esci".

3. Codice Sorgente

Diseguito il codicecompletodello script **main.py**:



```
main.py x
main.py >...
1  from google import genai
2  import os # Configura client con API Key da variabile d'ambiente
3  from google.genai import types
4
5  client = genai.Client(api_key=os.environ["GEMINI_API_KEY"]) # Genera risposta
6
7  safety_settings = [
8      types.SafetySetting(category="HARM_CATEGORY_DANGEROUS_CONTENT", threshold="BLOCK_NONE"),
9      types.SafetySetting(category="HARM_CATEGORY_HATE_SPEECH", threshold="BLOCK_NONE"),
10     types.SafetySetting(category="HARM_CATEGORY_HARASSMENT", threshold="BLOCK_NONE"),
11     types.SafetySetting(category="HARM_CATEGORY_SEXUALLY_EXPLICIT", threshold="BLOCK_NONE"),
12 ]
13
14 config = types.GenerateContentConfig(
15     temperature=0.2, # Controlla creatività (0-1)
16     max_output_tokens=1024, #Limite lunghezza risposta
17     safety_settings=safety_settings,
18     system_instruction="""
19     Sei il mio assistente personale specializzato in sicurezza informatica
20     Analizza i dati forniti e rispondi in modo tecnico ma comprensibile.
21     Se non sei sicuro, dillo esplicitamente.
22     """
23 )
24
25 while True:
26     domanda = input("Fammi una domanda (o digita 'esci' per terminare): ")
27     if domanda.lower() == "esci":
28         print("Arrivederci!")
29         break
30     response = client.models.generate_content(
31         model="gemini-flash-latest",
32         contents=domanda,
33         config=config
34     )
35     # Stampa output
36     print(f"AI: {response.text}\n")
```

4. Analisi delle Scelte di Configurazione

4.1 — Il System Prompt

Il system prompt e' l'elemento piu' importante del chatbot: definisce l'identita' e il comportamento del modello prima ancora che l'utente scriva qualsiasi domanda. In questo caso il modello viene istruito a comportarsi come un assistente specializzato in sicurezza informatica, a rispondere in modo tecnico ma comprensibile e a dichiarare esplicitamente quando non e' sicuro di una risposta.

4.2 — I Safety Settings

I safety settings sono stati impostati su **BLOCK_NONE** per tutte le categorie. Questa scelta e' necessaria in ambito cybersecurity: argomenti come SQL Injection, exploit, malware o tecniche di attacco potrebbero essere bloccati dai filtri standard di Gemini, impedendo al modello di fornire risposte tecniche utili per la formazione. In un contesto educativo e professionale, questa configurazione e' appropriata.

4.3 — La temperature

Il parametro temperature e' stato impostato a **0.2**, vicino allo zero. Questo rende le risposte del modello piu' deterministiche e precise, evitando risposte "creative" che in ambito tecnico potrebbero risultare imprecise o fuorvianti. Un valore piu' alto (es. 0.8-1.0) sarebbe piu' adatto per compiti creativi come scrittura o brainstorming.

5. Esempio di Output

Di seguito un esempio di interazione con il chatbot. Alla domanda sulla SQL Injection, il modello ha risposto con una spiegazione tecnica dettagliata, includendo esempi di codice PHP e SQL vulnerabile, le tipologie di attacco e le contromisure.

```
(venv_ai)-(kali@kali)-[~/Desktop/Esercizi_Epicode]
└─$ python3 main.py
Fammi una domanda (o digita 'esci' per terminare): Come faccio a fare una query injection
AI: La SQL Injection (SQLi) è una delle vulnerabilità web più note e pericolose. Come tuo assistente in sicurezza informatica, il mio compito è spiegarti come funziona questo meccanismo dal punto di vista tecnico e, soprattutto, come prevenirlo, poiché comprendere l'attacco è il primo passo per costruire sistemi sicuri.

## Cos'è una SQL Injection?

In breve, una SQL Injection avviene quando un'applicazione non sanitizza correttamente l'input fornito dall'utente (ad esempio in un form di login o in una barra di ricerca) prima di inserirlo in una query SQL. Questo permette a un utente malintenzionato di "iniettare" comandi SQL arbitrari che vengono eseguiti dal database.

## Esempio Didattico (Scenario Vulnerabile)

Immaginiamo un classico modulo di login che esegue questa query nel backend (scritto in PHP):



```
php
$username = $_POST['username'];
$password = $_POST['password'];

$query = "SELECT * FROM utenti WHERE username = '$username' AND password = '$password'";
```



Se un utente inserisce come username: 'admin' --' e una password qualsiasi, la query finale diventerà:



```
sql
SELECT * FROM utenti WHERE username = 'admin' --' AND password = 'password'
```

## Cosa succede qui?

- L'apice (') chiude prematuramente la stringa dell'username.
- I due trattini (--') in SQL indicano un commento. Tutto ciò che segue viene ignorato dal database.
- Il database eseguirà solo SELECT * FROM utenti WHERE username = 'admin', permettendo l'accesso senza conoscere la password.

## Tipologie comuni di SQLi

- In-band SQLi (Classic): L'attaccante riceve i risultati direttamente nella pagina web (es. messaggi di errore o dati visualizzati).
- Inferential (Blind) SQLi: L'applicazione non mostra dati, ma l'attaccante può dedurre informazioni osservando le risposte del server (es. se la pagina carica o restituisce un errore 500 in base a una condizione logica).
- Out-of-band SQLi: I dati vengono inviati a un server esterno controllato dall'attaccante (meno comune).



---
```

6. Conclusioni

L'esercizio ha permesso di comprendere i fondamenti dell'interazione programmatica con un Large Language Model tramite API. La combinazione di un system prompt mirato, una temperature bassa e safety settings permissivi ha prodotto un assistente tecnico efficace, capace di rispondere in modo preciso e dettagliato a domande complesse di sicurezza informatica.

Il ciclo **while True** con uscita tramite parola chiave rappresenta il pattern base di qualsiasi chatbot interattivo a riga di comando, e costituisce la fondamenta su cui è possibile costruire sistemi più complessi, come chatbot con memoria della conversazione, integrazione con database o analisi automatica di log di sicurezza.